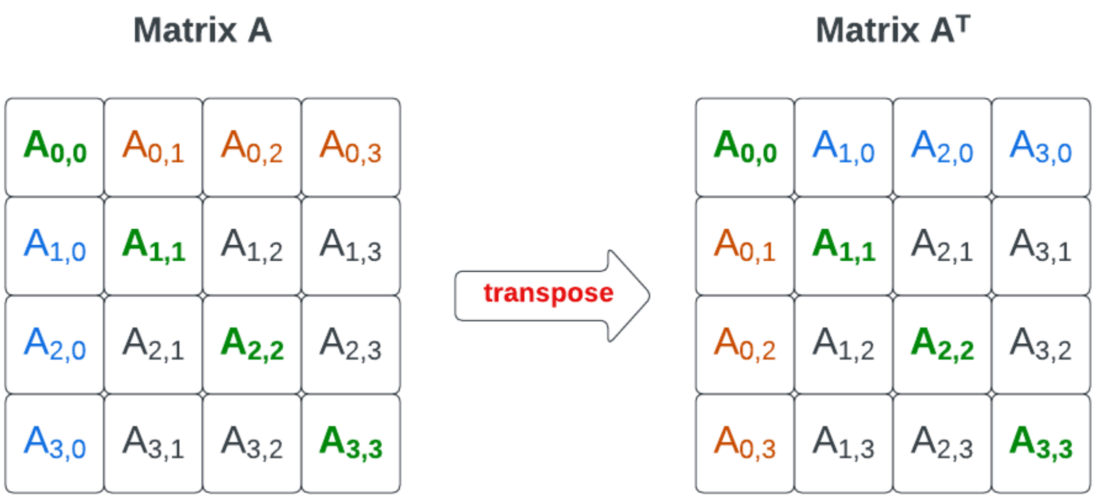
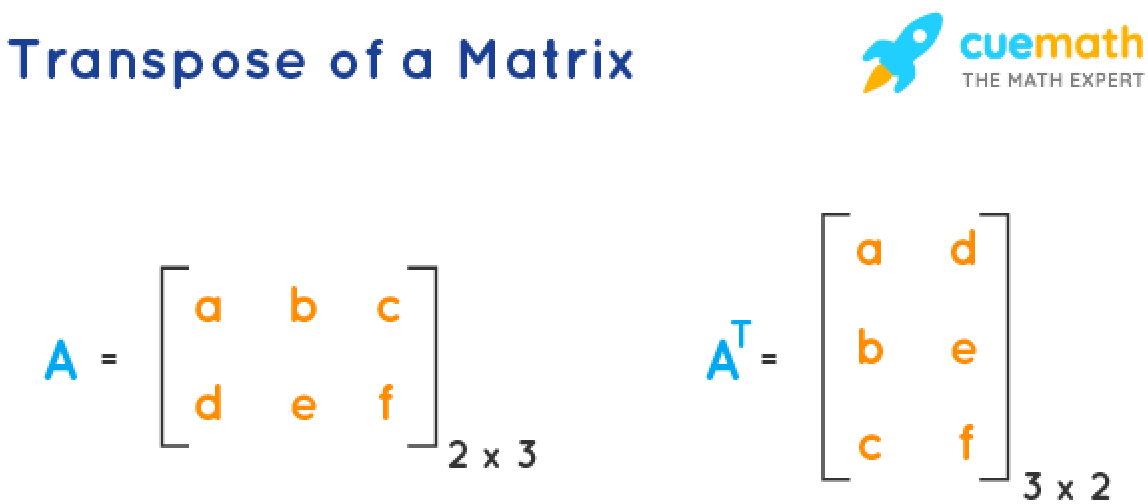
Transpose of a Matrix (C Programming)





1. Definition

The **transpose of a matrix** is obtained by **interchanging rows and columns**.

If the original matrix is **A**, then its transpose is **A^T**.

Rule

$A[i][j] \rightarrow A^T[j][i]$

Meaning:

- Row becomes column
- Column becomes row

2. Example

Original Matrix **A (2×3)**

1 2 3
4 5 6

Transpose Matrix **A^T (3×2)**

1 4
2 5
3 6

So,

Row → Column
Column → Row

3. Algorithm for Transpose

Step-by-step process:

1. Read number of **rows and columns**.
2. Input the matrix elements.
3. Use nested loops.
4. Assign:

transpose[j][i] = matrix[i][j]

5. Print the transpose matrix.

4. C Program for Transpose of a Matrix

```
#include<stdio.h>

int main()
{
    int matrix[10][10], transpose[10][10];
    int r, c, i, j;

    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);

    printf("Enter matrix elements:\n");
    for(i = 0; i < r; i++)
    {
        for(j = 0; j < c; j++)
        {
            scanf("%d", &matrix[i][j]);
        }
    }

    for(i = 0; i < r; i++)
    {
        for(j = 0; j < c; j++)
        {
            transpose[j][i] = matrix[i][j];
        }
    }

    printf("\nTranspose Matrix:\n");

    for(i = 0; i < c; i++)
    {
        for(j = 0; j < r; j++)
        {
            printf("%d ", transpose[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

5. Program Explanation

Step 1: Declare matrices

```
int matrix[10][10], transpose[10][10];
```

One for original matrix and another for transpose.

Step 2: Input matrix

```
for(i=0;i<r;i++)
{
   for(j=0;j<c;j++)
   scanf("%d",&matrix[i][j]);
}
```

Step 3: Transpose logic

```
transpose[j][i] = matrix[i][j];
```

This swaps **row and column**.

Step 4: Print transpose

Rows and columns are reversed.

6. Example Execution

Input

Rows = 2
Columns = 3

Matrix:
1 2 3
4 5 6

Output

Transpose Matrix:

1 4
2 5

3 6

7. Special Case (Square Matrix)

If matrix is **n × n**, transpose keeps the same size.

Example:

Original

1 2
3 4

Transpose

1 3
2 4

8. Time Complexity

Operation	Complexity
Transpose	**O(n × m)**

Because every element must be visited once.